

Encontro 1: Introdução ao Controle de Versionamento, Git e GitHub

Sumário

Encontro 1: Git e GitHub.....	1
1. Por que o versionamento é importante?	2
1.1. Vantagens.....	2
2. O que são o Git e o GitHub?	3
3. Explicando o Git.....	3
4. Explicando o GitHub	3
5. Diferença entre Git e GitHub	4
6. Instalação e Configuração do Git.....	4
7. Criando Conta e Repositórios no GitHub	4
8. Fluxo de Trabalho Básico com Git	4
9. Atualizando Repositório no GitHub	5
10. Situações comuns	5
10.1. Configurar credenciais locais para acesso ao GitHub (HTTPS e SSH).....	5
10.2. Criar um repositório local e enviar para o GitHub (HTTPS)	6
10.3. Enviar para o GitHub usando SSH (alternativa).....	6
10.4. Baixar (clonar) um repositório do GitHub, alterar localmente e enviar de volta	6
10.5. Trazer atualizações do GitHub para sua máquina (pull).....	7
10.6. Erros comuns e como resolver rápido	7
11. Conclusão.....	7

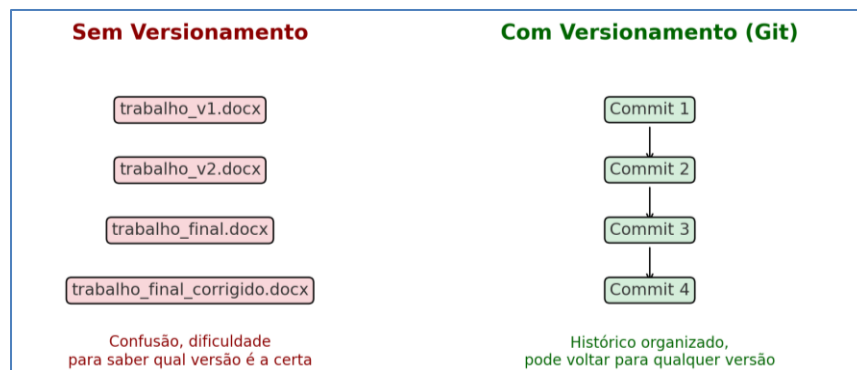
1. Por que o versionamento é importante?

Imagine que você está escrevendo um trabalho da escola ou da faculdade. A cada mudança importante, você salva uma cópia do arquivo:

- trabalho_v1.docx
- trabalho_v2.docx
- trabalho_final.docx
- trabalho_final_corrigido.docx

Com o tempo, fica confuso lembrar qual é a versão certa. Além disso, se você apagar um arquivo por engano ou precisar voltar a uma versão anterior, pode ser difícil recuperar.

O **versionamento de código** resolve exatamente esse problema, mas de uma forma organizada e inteligente.



Evolução de um trabalho na forma sem versionamento e com versionamento

1.1. Vantagens

- **Guardar cada mudança:** Toda vez que você faz uma alteração no código, o sistema registra essa mudança em um histórico. Assim, você pode voltar atrás a qualquer momento.
- **Trabalhar em equipe sem bagunça:** Se várias pessoas estão mexendo no mesmo projeto, cada uma pode trabalhar em sua parte sem atrapalhar a outra. Depois, as alterações são reunidas de forma controlada.
- **Segurança contra erros:** Se algo der errado em uma nova versão do código, é possível retornar a um ponto em que tudo funcionava bem, sem perder o progresso feito.
- **Organização:** Você pode separar versões de teste das versões oficiais do projeto. Isso evita que mudanças inacabadas quebrem o programa que já está funcionando.
- **Acompanhamento e aprendizado:** Como tudo fica registrado, é possível entender como o projeto foi construído, quem fez cada parte e em que momento.

O versionamento é como ter um **histórico completo do seu projeto**, que garante segurança, organização e facilita o trabalho em grupo.

2. O que são o Git e o GitHub?

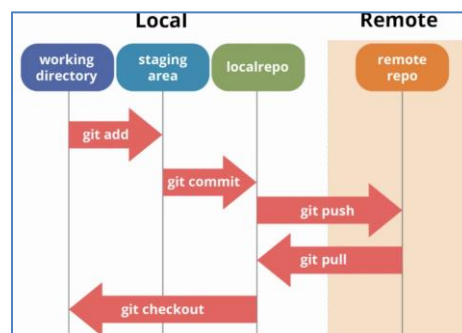
O Git e o GitHub são ferramentas indispensáveis para o programador web moderno. O Git é um sistema de controle de versão que permite acompanhar e gerenciar as alterações em arquivos e projetos. Já o GitHub é uma plataforma que hospeda repositórios Git e adiciona recursos de colaboração, tornando mais fácil trabalhar em equipe e compartilhar projetos com o mundo.

3. Explicando o Git

O Git é um sistema de controle de versão distribuído, criado por Linus Torvalds em 2005. Ele permite que cada desenvolvedor mantenha uma cópia completa do projeto com todo o histórico de mudanças, facilitando a colaboração e a recuperação de versões anteriores.

Conceitos principais do Git:

- Repositório: local onde o histórico de versões do projeto é armazenado.
- Commit: registro de uma alteração feita nos arquivos.
- Branch: ramificação usada para desenvolver novas funcionalidades de forma isolada.
- Merge: processo de unir alterações feitas em diferentes branches.
- Clone: cópia de um repositório existente para a máquina local.
- Push: envio das alterações locais para o repositório remoto.
- Pull: atualização do repositório local com as mudanças do remoto.



Resumo das ações relacionadas a controle de versionamento



Estágios fundamentais do GIT

4. Explicando o GitHub

O GitHub é uma plataforma online baseada em Git que oferece recursos adicionais de colaboração, como controle de permissões, gerenciamento de projetos e integração com ferramentas de desenvolvimento. É amplamente utilizado para hospedar projetos de código aberto e privados.

Com o GitHub é possível:

- Criar repositórios públicos e privados.
- Compartilhar e colaborar em projetos com outros desenvolvedores.
- Contribuir em projetos de código aberto.
- Criar páginas estáticas com o GitHub Pages.
- Documentar projetos usando arquivos README.

5. Diferença entre Git e GitHub

Embora relacionados, Git e GitHub são diferentes:

- Git: é o software de controle de versão, usado localmente.
- GitHub: é a plataforma online que hospeda repositórios Git e oferece ferramentas de colaboração.

6. Instalação e Configuração do Git

Para instalar e configurar o Git:

1. Acesse <https://git-scm.com/> e baixe a versão para seu sistema operacional.
2. Siga o assistente de instalação.
3. Configure seu nome e e-mail no terminal:

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
```
4. Para ver as configurações globais do Git:

```
git config --list --show-origin
```

7. Criando Conta e Repositórios no GitHub

Passo a passo para criar uma conta:

1. Acesse <https://github.com/>.
2. Clique em "Sign up" e preencha e-mail, senha e nome de usuário.
3. Confirme sua conta por e-mail e faça login.

Passo a passo para criar um repositório no GitHub:

1. Clique em "New" ou no botão "+ > New repository".
2. Escolha um nome e adicione uma descrição.
3. Defina como público ou privado.
4. (Opcional) Inicie com um README.
5. Clique em "Create repository".

8. Fluxo de Trabalho Básico com Git

Passos principais para usar o Git em conjunto com o GitHub:

1. Crie uma pasta para o projeto e abra o terminal nela.
2. Inicialize o repositório:

```
git init
```
3. Adicione os arquivos:

```
git add .
```
4. Registre a alteração:

```
git commit -m "Mensagem do commit"
```
5. No GitHub, crie um repositório vazio.

6. Conecte o repositório local ao remoto:

```
git remote add origin https://github.com/seuusuario/nomedorepositorio.git
```

7. Envie os arquivos:

```
git push -u origin main
```

9. Atualizando Repositório no GitHub

Depois de configurar o repositório, atualize sempre que fizer mudanças:

1. Salve as alterações nos arquivos.

2. Adicione as alterações:

```
git add .
```

3. Registre as mudanças:

```
git commit -m "Descrição das alterações"
```

4. Envie para o GitHub:

```
git push
```

10. Situações comuns

10.1. Configurar credenciais locais para acesso ao GitHub (HTTPS e SSH)

Configuração mínima do Git (nome e e-mail):

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seuemail@exemplo.com"
```

Opção A – HTTPS com Token de Acesso Pessoal (PAT):

- No GitHub, vá em Settings > Developer settings > Personal access tokens > Tokens (classic) > Generate new token.
- Marque os escopos necessários (por exemplo: repo). Copie o token gerado (ele só aparece uma vez).
- No primeiro push via HTTPS, informe seu usuário do GitHub e, como senha, cole o token (PAT).
- Para gravar credenciais automaticamente:
 - Windows:

```
git config --global credential.helper manager
```
 - macOS:

```
git config --global credential.helper osxkeychain
```
 - Linux:

```
git config --global credential.helper store
```

 (salva em texto plano)

Opção B – SSH (recomendado a longo prazo):

- Gere uma chave:

```
ssh-keygen -t ed25519 -C "seuemail@exemplo.com"
```

 (Enter para aceitar o local padrão).
- Inicie o agente e adicione a chave:
 - Windows (PowerShell):

```
Start-Service ssh-agent ; ssh-add $HOME/.ssh/id_ed25519
```
 - macOS/Linux (bash):

```
eval "$(ssh-agent -s)" ; ssh-add ~/.ssh/id_ed25519
```
- Copie a chave pública (~/.ssh/id_ed25519.pub) e ponha em GitHub>Settings>SSH and GPG keys>New SSH key.
- Teste a conexão:

```
ssh -T git@github.com
```

 (responda yes se solicitado).
- Use a URL SSH do repositório (git@github.com:usuario/repo.git).

10.2. Criar um repositório local e enviar para o GitHub (HTTPS)

1. Crie uma pasta para o projeto e entre nela pelo terminal.
2. Inicialize o repositório:
`git init`
3. Crie um arquivo README.md e outros arquivos do projeto (ex.: index.html).
4. Veja o status:
`git status`
5. Adicione os arquivos:
`git add .`
6. Faça o primeiro commit:
`git commit -m "Primeiro commit"`
7. No GitHub (web), crie um repositório vazio (sem README para evitar conflito no primeiro push).
8. Conecte o remoto (HTTPS):
`git remote add origin https://github.com/seuusuario/nomedorepositorio.git`
9. Defina a branch principal como main (se necessário):
`git branch -M main`
10. Envie para o GitHub:
`git push -u origin main`

10.3. Enviar para o GitHub usando SSH (alternativa)

Após configurar a chave SSH (10.1), conecte o remoto por SSH:

```
git remote remove origin (se já existir um remoto HTTPS e você quiser trocar)
git remote add origin git@github.com:seuusuario/nomedorepositorio.git
git push -u origin main
```

10.4. Baixar (clonar) um repositório do GitHub, alterar localmente e enviar de volta

1. No GitHub, copie a URL do repositório (HTTPS ou SSH).
2. No terminal, escolha uma pasta de trabalho e execute:
 - HTTPS:
`git clone https://github.com/usuario/nomedorepositorio.git`
 - SSH:
`git clone git@github.com:usuario/nomedorepositorio.git`
3. Entre na pasta clonada:
`cd nomedorepositorio`
4. Crie/edite arquivos e salve as mudanças.
5. Veja o que mudou:
`git status`
6. Adicione as alterações:
`git add .`
7. Registre as mudanças:
`git commit -m "Minha atualização"`
8. Envie para o GitHub:
`git push`

10.5. Trazer atualizações do GitHub para sua máquina (pull)

1. Entre na pasta do repositório local.
2. Baixe e mescle as mudanças do remoto:
`git pull`
3. Se houver conflitos, o Git avisará quais arquivos precisam ser resolvidos.
4. Edite os arquivos conflitantes e marque a resolução:
`git add <arquivo_resolvido>`
`git commit -m "Resolve conflitos"`

10.6. Erros comuns e como resolver rápido

- `remote origin already exists` → Ajuste ou remova e recrie o remoto:
`git remote set-url origin <URL>` ou
`git remote remove origin`
- `rejected non-fast-forward` → Puxe primeiro e depois envie:
`git pull --rebase` (resolva conflitos) e então
`git push`
- Permissão negada (publickey) → Configure SSH (10.1 Opção B) ou use HTTPS com token (Opção A).
- A branch padrão é master e você quer main:
`git branch -M main` (ajuste no GitHub se necessário)

11. Conclusão

O Git e o GitHub são fundamentais para a prática do programador web. Com o Git, você organiza e versiona projetos; com o GitHub, colabora e compartilha. Com este material, o estudante tem a base necessária para iniciar o uso dessas ferramentas em seus projetos.